

Machine Learning: What Could Go Wrong?

Markus M. Geipel

 mgeipel.github.io  linkedin.com/in/markus-geipel



September 6, 2026

1 Machine Learning Models in the Wild

There are things that can go wrong with machine learning models. The danger is not that they behave in a super-intelligent way and threaten you and humanity. At least not in 2026 and not for models you train with a budget below the ones of frontier labs. The problem is rather that these models might not do what you think they should do, or that they simply behave stupidly in unexpected ways. Forewarned is forearmed.

2 Failure Cases

Let us look at typical problems and how to address them . . .

2.1 Problem Definition Exploits

Machine learning models and computers alike are “mathematical creatures”. They take quite literally the data and definitions you give them. If you define an optimisation objective, the model will optimise for it and it alone[1]. Imagine you want a machine learning model to improve the design of a car. The objective is to consume less energy or fuel. If that is all you put in the objective, the model will give you a car that does not drive, as such a car does not consume energy at all.

The same is true for simulation-based optimisation. Let us say you want to learn perfect movement control for a robot. As countless hours of experience are needed, this is faster done in a simulation, not the real world. If the simulation happens to have a bug, a rounding error, or any kind of glitch that can be exploited by the

[1]: Amodei et al. (2016), *Concrete Problems in AI Safety*

* This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License. <https://creativecommons.org/licenses/by-nc-nd/4.0/>

robot to move better, a good machine learning model will exploit it. It cannot know that the bug was unintended. For the model, the bug is part of the problem definition. This kind of specification gaming has been observed across dozens of real systems[2].

The bottom line is that **problem definitions, quality metrics and optimisation objectives have to be defined very carefully**. The dialogue between domain experts and machine learning experts is essential. Just like any other piece of software, **machine learning algorithms and models should be tested thoroughly**. You should have test cases. They not only help you to minimise the risk of problem-definition exploits but are also useful in discussions between domain and ML experts. Test cases can become part of the specification.

2.2 Skew, Errors, and Bias in the Data

Logical inference on wrong data will produce wrong results, and machine learning applied to wrong data will make wrong predictions. **Narrow AI** does not possess the necessary common sense to look at the results and say, “wait a minute, that can’t be right!” **It is up to the expert to add safeguards.**

Apart from plain errors there are two related problems, which are more subtle, if not downright devious: skew and bias.

2.2.1 Skew

Skew means that some predictions or classifications are severely underrepresented in the data. For example, in visual quality inspection of products, it is quite common that the overwhelming majority of images show flawless products and only a tiny fraction of images show defective products. Say 99.99% are flawless and 0.01% are defective (not even close to six sigma!). If you measure the quality of a machine learning model detecting defects as a percentage of correct classifications (accuracy), the model may easily achieve an accuracy of 99.99%, simply by classifying everything as correct. **An accuracy of 99.99% sounds great but is entirely useless in this case.** Be aware of this pitfall!

Here is a recipe to deal with skew: First, make sure to detect skew in the data if present. This can easily be done with a simple class distribution plot. Second, adjust your quality metrics. In our example this would mean moving away from accuracy and giving false positives (faulty products classified as correct) more weight compared to false negatives (correct products classified as faulty). Depending on your specific use case, the adjustment might look different, though. Third, the skew can often be counterbalanced by resampling the data or applying more data augmentation to underrepresented classes.

[2]: Krakovna et al. (2020), *Specification Gaming: The Flip Side of AI Ingenuity*

On two occasions I have been asked, ‘Pray, Mr. Babbage, if you put into the machine wrong figures, will the right answers come out?’ ... I am not able rightly to apprehend the kind of confusion of ideas that could provoke such a question.

— Charles Babbage, *Passages from the Life of a Philosopher*

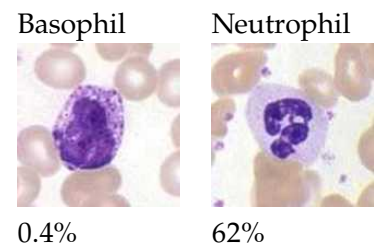


Figure 1: If you want to classify white blood cells, you will encounter skew. In a normal sample of blood you will find 62% neutrophils, but only 0.4% basophils. Basophils will be heavily underrepresented in your training data. Images public domain via Wikimedia Commons.

2.2.2 Bias

The point of machine learning is that it learns whatever is in the training data. This can become a problem when you are not aware of the nature of the data. In 2016 Microsoft launched the machine-learning-based chatbot Tay and claimed: “*The more you chat with Tay the smarter she gets.*” To their chagrin, Tay turned out to become quite racist as it learned from biased Twitter data: The Guardian wrote *Tay, Microsoft’s AI chatbot, gets a crash course in racism from Twitter*[3]. While ChatGPT does better than Tay in this respect, it is still full of biases[4].

Make sure you know your data! Bias can be less obvious than the blatant racism in tweets. If a machine learning model is trained to assess credit applications, and past applications used as training data discriminated against certain social groups, so will the machine learning model. **It will perpetuate the bias and injustice hidden in the data.** It is up to you to understand and clean the data before giving it to the model.

2.3 Blind Extrapolation

Imagine you have empirically measured the relationship between two properties of a system. Let us say, for instance, voltage and current of a transistor. With this data you fit a model. For the sake of simplicity, we use linear regression, but it could also be a neural network, a K-nearest-neighbour model, or a support vector machine. The result of our experiment is shown in Figure 2. The black dots are the measurements, the blue line shows our linear model.

Note that we can ask for predictions for any value of x and get a y as answer. The model simply extrapolates and never responds “I don’t know”. We see in the figure that only part of the model is backed by data. Please note that the extrapolation might be very wrong.

In the case of a transistor, at a certain voltage the relationship with current will not be linear anymore: the transistor will simply burn out. It is highly likely that you will not see this situation in the data because experimenters are not inclined to destroy the system they are measuring, especially in industrial settings where such systems tend to be expensive.

The important lesson is to **mistrust extrapolations that are not backed by training data!** Note that each type of machine learning model has its own way of extrapolating. There are models that give you a confidence value along with their prediction. Still, caution is advised: most of them are vastly overconfident. Knowing when one does not know is an art in which neither humans nor machines have a convincing track record.

[3]: Hunt (2016), *Tay, Microsoft’s AI Chatbot, Gets a Crash Course in Racism from Twitter*

[4]: Abid et al. (2021), *Persistent Anti-Muslim Bias in Large Language Models*

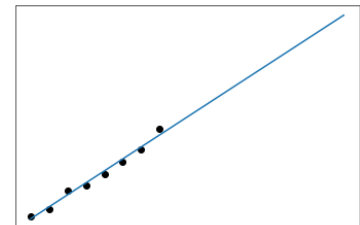


Figure 2: Linear regression extrapolating beyond the data.

A keen observer might note that blind extrapolation is also common among humans. In fact, the problem is even deeper. In *The Logic of Scientific Discovery* [5], Karl Popper criticised the traditional scientific method of induction, where general laws are inferred based on empirical observations. He argued that induction can never conclusively verify a theory, only falsify it. Nassim Nicholas Taleb, in his book *The Black Swan* [6], raises a similar point.

2.4 Adversarial Input

When a machine learning model, or any type of software for that matter, is fed data specifically selected or manipulated to make the model or software fail, we speak of adversarial input. It has been shown that deep neural networks are susceptible to slightly and purposefully manipulated images, causing misclassifications. Most strikingly, the manipulation may not even be visible to the human eye. To learn more, take a look at the OpenAI blog post *Attacking Machine Learning with Adversarial Examples*[7].

The problem is not limited to deep learning. Classical spam filters used a much simpler algorithm called Naive Bayes, and spammers adapted their spam to avoid detection. As soon as the model leaves your R&D lab, there is a real possibility that an attacker may try to exploit it with adversarial input.

The first step is to **make a risk assessment**. Imagine that attackers can fool your machine learning model. What could they achieve by doing it? Could they . . .

- ▶ elevate their privileges?
- ▶ steal data?
- ▶ manipulate data?
- ▶ delete data?
- ▶ pretend to be someone else?
- ▶ make your product or service unusable?

Once you have an answer to these questions, you can start to analyse the potential impact. Based on this risk assessment, countermeasures can be planned in a principled fashion. As already indicated, adversarial input is not a problem specific to machine learning but a general cyber security risk. If the application, machine, or product using machine learning is safety- or security-critical, **do not hesitate to involve a cyber security expert**.

2.5 Missing Guarantees

When working with machine deduction, where rules and knowledge are explicitly and formally defined, guarantees are easy. When building machine learning models, it is hard to guarantee certain constraints. You can encode constraints in the error metrics of some machine learning methods, but this still does not guarantee that the constraint will be respected with new data.

The pragmatic way to make the model safe is to **sandbox it**. This means that you build code around the model which enforces constraints—if the control action must always stay below 1, for instance. A very simple sandbox could look like Listing 1.

You evaluate the model, but before using the model output, you check its validity and correct it if necessary. This way the model does not interact with the outside world directly, but runs in a sandbox. It is also common to write code to sanitise inputs before feeding them to the model. **A machine learning model does not eliminate the need to write code**. Machine learning models form

[7]: OpenAI (2017), *Attacking Machine Learning with Adversarial Examples*

Listing 1: Sandboxing the model so that the action is guaranteed to be below 1.

```
# get the model output
action = model(input)

# hand-coded sandbox
if action > 1:
    action = 1

# use the model output
machine.setAction(action)
```

part of software that is designed and implemented just like any other piece of software. There should be documented requirements and test cases. If the requirements include guarantees, they need to be enforced by the code around the model, and they need to be properly tested.

There are situations where it is hard to sandbox the model. For instance if the model is the source of a safety critical signal as in obstacle detection in autonomous driving. In these cases the signal should be generated by a number of independent models or techniques, followed by a consensus building algorithm. But this is beyond the scope of this blog post.

2.6 Concept Shift

Concepts and system dynamics change over time. Data and models become outdated. To take it to the extreme, imagine the Roman emperor Augustus trying to understand the social media posts of modern-day Romans. Concepts and the language itself changed entirely. A hypothetical language model created in his time would be rather worthless for today's data. You might say: I am not working with languages but with machines. Does this still apply? Absolutely. **The dynamics of machines change over time:** there is wear and tear, they undergo maintenance, parts are replaced, and so on. Do not expect your models to keep working well once the machine has changed.

There is nothing you can do to prevent your models from becoming outdated. You can, however, detect when that happens and act accordingly. First, in many circumstances you can track the performance of your model and detect a deterioration in performance. Imagine you have a machine learning model predicting, let us say, the energy consumption of a factory. You can compare the prediction with the later measured actual consumption. Second, if you cannot measure the quality directly, you can still analyse the input data: do statistical properties of it change? If you do visual inspection, you could check if the image brightness and saturation change over time. This could mean that light conditions are changing—possibly to a degree that your model is not prepared to account for. Finally, you can keep track of the model's classifications or predictions and check if they change statistically. Imagine you have a model that classifies products as OK or damaged. Track the percentage of products being classified as OK per day. If you see a drift in this percentage, either something is going on in production, or your model is not adequate anymore. Either way, it is time to take a closer look. All these techniques are **canaries in the data mine**. They warn you that something might be off with your model.

Word meanings keep drifting, and not just once. *Nice* [8]:

- ▶ Latin *nescius*: “ignorant”
- ▶ Old/Middle English: “foolish, silly”
- ▶ 15th–16th c.: “fussy, over-precise”
- ▶ Today: “pleasant”

A near-total reversal in meaning, in at least four separate hops.

3 Final Thoughts

We have seen that building and using machine learning models requires engineering discipline. Just as with other engineering artefacts we have to check what can go wrong during creation (Problem Definition and Skew, Errors, and Bias in the Data). We have to check what goes in (Adversarial Input and Concept Shift) and what comes out (Missing Guarantees). And finally, we have to be aware of the conditions under which the model can reliably operate (Blind Extrapolation). This is not an exhaustive survey of failure cases in machine learning — these are simply the most typical ones I have witnessed. Taking them into account is a big step towards machine learning models that work in practice and not just in a lab prototype. Have fun training models!

Bibliography

- [1] Dario Amodei et al. *Concrete Problems in AI Safety*. arXiv:1606.06565. June 2016. URL: <https://arxiv.org/abs/1606.06565> (cited on page 1).
- [2] Victoria Krakovna et al. *Specification Gaming: The Flip Side of AI Ingenuity*. Apr. 2020. URL: <https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/> (cited on page 2).
- [3] Elle Hunt. *Tay, Microsoft's AI Chatbot, Gets a Crash Course in Racism from Twitter*. Mar. 2016. URL: <https://www.theguardian.com/technology/2016/mar/24/tay-microsofts-ai-chatbot-gets-a-crash-course-in-racism-from-twitter> (cited on page 3).
- [4] Abubakar Abid, Maheen Farooqi, and James Zou. 'Persistent Anti-Muslim Bias in Large Language Models'. In: *Proceedings of the 2021 AAAI/ACM Conference on AI, Ethics, and Society*. AIES '21. 2021, pp. 298–306. DOI: [10.1145/3461702.3462624](https://doi.org/10.1145/3461702.3462624) (cited on page 3).
- [5] Karl R. Popper. *The Logic of Scientific Discovery*. Translation of *Logik der Forschung* (1934). London: Hutchinson, 1959 (cited on page 3).
- [6] Nassim Nicholas Taleb. *The Black Swan: The Impact of the Highly Improbable*. New York: Random House, 2007 (cited on page 3).
- [7] OpenAI. *Attacking Machine Learning with Adversarial Examples*. Feb. 2017. URL: <https://openai.com/index/attacking-machine-learning-with-adversarial-examples/> (cited on page 4).
- [8] Douglas Harper. *Nice*. Online Etymology Dictionary; archived snapshot, since the live URL currently redirect-loops. 2001. URL: <https://web.archive.org/web/20260810151120/https://www.etymonline.com/word/nice> (visited on 08/10/2026) (cited on page 5).